StayEase

Hotel Booking System

Microservice-Based Cloud Application

SLIIT | Department of Computer Science & Software Engineering

Module: Current Trends in Software Engineering (SE4010)
Cloud Computing Assignment | 2026 | Semester 1

Student	Service Assigned	Repository
Student 1	Auth Service	stayease-auth-service
Student 2	Hotel Listing Service	stayease-hotel-service
Student 3	Booking Service	stayease-booking-service
Student 4	Payment & Notification Service	stayease-payment-service

1. Project Overview

StayEase is a cloud-native hotel booking system built using a microservice architecture. The application is broken down into four independently deployable microservices, each owned by one group member. Together, they form a complete, end-to-end hotel booking platform.

1.1 Application Summary

StayEase allows users to register and log in, browse available hotels and rooms, make bookings, and receive payment confirmations. The system is designed to demonstrate modern cloud computing and DevOps practices.

1.2 Technology Stack

Layer	Technology
Language / Framework	Node.js with Express.js
Database	MongoDB Atlas (Free Tier)
Containerization	Docker
Container Registry	Docker Hub
CI/CD Pipeline	GitHub Actions
Cloud Provider	AWS (ECS + ECR) / Azure Container Apps
Security Scanning	SonarCloud (Free Tier)
API Documentation	OpenAPI / Swagger
Token Authentication	JSON Web Tokens (JWT)

2. System Architecture

The system follows a microservices architecture pattern where each service is independently developed, containerized, deployed, and maintained. All services communicate over HTTP REST APIs.

2.1 Architecture Diagram Description

The high-level architecture consists of:

- An API Gateway acting as the single entry point for all client requests
- Four microservices deployed independently on cloud infrastructure
- Each microservice has its own dedicated MongoDB Atlas database (database-per-service pattern)
- Services communicate synchronously via HTTP REST APIs
- GitHub Actions handles automated CI/CD for each service
- SonarCloud performs static code analysis on every push

2.2 Inter-Service Communication Flow

The following describes how the four services communicate with each other:

From Service	To Service	Purpose	Trigger
Booking Service	Auth Service	Verify JWT token	Every incoming request
Booking Service	Hotel Listing Service	Check room availability	When creating a booking
Booking Service	Payment Service	Trigger payment processing	After booking is confirmed
Payment Service	Notification (internal)	Send confirmation email/SMS	After successful payment

2.3 GitHub Organization Structure

```
Organization: stayease-sliit-2026
├── stayease-auth-service        (Student 1)
├── stayease-hotel-service       (Student 2)
├── stayease-booking-service     (Student 3)
└── stayease-payment-service     (Student 4)
```

3. Microservice Descriptions

3.1 Auth Service (Student 1)

The Auth Service is responsible for user identity management. It handles registration, login, and JWT-based token verification. All other services call this service to validate incoming requests.

Endpoints:

Method	Endpoint	Description	Auth Required
POST	/auth/register	Register a new user account	No
POST	/auth/login	Login and receive JWT token	No
GET	/auth/verify	Validate a JWT token (called by other services)	Yes
GET	/auth/profile	Get current user profile	Yes
PUT	/auth/profile	Update user profile	Yes
POST	/auth/logout	Logout and invalidate token	Yes

Database Schema (Users Collection):

```
{ id, name, email, passwordHash, role: ['guest','admin'], createdAt, updatedAt
}
```

Integration Points:

- Called by: Booking Service (GET /auth/verify) to validate JWT on every request
- Called by: Hotel Service (GET /auth/verify) to protect admin endpoints
- Called by: Payment Service (GET /auth/verify) to validate user identity

---

3.2 Hotel Listing Service (Student 2)

The Hotel Listing Service manages all hotel and room data. It provides endpoints for listing, searching, filtering, and managing hotels and their rooms.

Endpoints:

Method	Endpoint	Description	Auth Required
GET	/hotels	List all hotels with optional filters	No
GET	/hotels/:id	Get single hotel details	No
POST	/hotels	Add a new hotel (admin only)	Yes (Admin)
PUT	/hotels/:id	Update hotel information	Yes (Admin)
DELETE	/hotels/:id	Delete a hotel	Yes (Admin)
GET	/hotels/:id/rooms	List all rooms in a hotel	No

Method	Endpoint	Description	Auth Required
POST	/hotels/:id/rooms	Add a new room to a hotel	Yes (Admin)
PUT	/hotels/:id/rooms/:roomId	Update room details	Yes (Admin)

Database Schema:

```
Hotel: { id, name, location, description, rating, images[], amenities[] }
Room:  { id, hotelId, type, price, capacity, isAvailable }
```

Integration Points:

- Calls: Auth Service (GET /auth/verify) to protect admin routes
- Called by: Booking Service (GET /hotels/:id/rooms) to check availability

---

3.3 Booking Service (Student 3)

The Booking Service is the core of the application. It manages the full booking lifecycle — from creation to confirmation to cancellation. It communicates with both the Auth Service and Hotel Listing Service.

Endpoints:

Method	Endpoint	Description	Auth Required
POST	/bookings	Create a new booking	Yes
GET	/bookings/:id	Get booking by ID	Yes
GET	/bookings/user/:userId	Get all bookings for a user	Yes
PUT	/bookings/:id/confirm	Confirm a booking	Yes
PUT	/bookings/:id/cancel	Cancel a booking	Yes
GET	/bookings/availability	Check room availability by date	No

Database Schema:

```
Booking: { id, userId, hotelId, roomId, checkIn, checkOut,
          status: ['pending','confirmed','cancelled'], totalPrice, createdAt }
```

Integration Points:

- Calls: Auth Service (GET /auth/verify) — validates JWT token on every request
- Calls: Hotel Service (GET /hotels/:id/rooms) — checks room availability before booking
- Calls: Payment Service (POST /payments) — triggers payment after booking creation

---

3.4 Payment & Notification Service (Student 4)

The Payment & Notification Service handles all financial transactions and user communications. It is triggered by the Booking Service and sends confirmations via email or SMS after successful payment.

Endpoints:

Method	Endpoint	Description	Auth Required
POST	/payments	Process a payment for a booking	Yes
GET	/payments/:bookingId	Get payment status by booking ID	Yes
POST	/payments/refund	Issue a refund	Yes
POST	/notifications/send	Send email or SMS notification	Internal
GET	/notifications/history	Get notification history for a user	Yes

Database Schema:

```
Payment: { id, bookingId, userId, amount, currency,
          status: ['pending','success','failed','refunded'], transactionId,
createdAt }
```

Integration Points:

- Calls: Auth Service (GET /auth/verify) — validates user identity
- Called by: Booking Service (POST /payments) — triggered after booking confirmation
- Internally triggers notification sending after successful payment

4. DevOps Practices

4.1 Version Control Strategy

Each microservice has its own public GitHub repository under the shared organization stayease-sliit-2026. All repositories follow the same branching strategy:

Branch	Purpose
main	Production-ready code only. Protected branch requiring PR approval.
develop	Integration branch for combining features before release.
feature/*	Individual feature development branches (e.g., feature/login-endpoint).

4.2 CI/CD Pipeline (GitHub Actions)

Each repository has an automated CI/CD pipeline defined in .github/workflows/ci.yml that runs on every push to main or develop branches.

Pipeline Stages:

Stage	Tool	Action
1. Checkout	GitHub Actions	Pull latest source code
2. Install	npm	Install project dependencies
3. Test	Jest	Run unit and integration tests
4. SAST Scan	SonarCloud	Static code security analysis
5. Build Image	Docker	Build container image
6. Push Image	Docker Hub	Push image to container registry
7. Deploy	AWS ECS / Azure	Deploy updated container to cloud

4.3 Containerization

Each microservice is containerized using Docker. A Dockerfile is included in the root of every repository. The container image is pushed to Docker Hub and consumed by the cloud deployment.

Dockerfile Structure (same for all services):

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --production
COPY . .
EXPOSE 3000
```

```
CMD ["node", "src/index.js"]
```

5. Security Measures

5.1 Application-Level Security

Security Measure	Implementation	Applies To
JWT Authentication	All protected endpoints validate Bearer tokens via Auth Service	All Services
Password Hashing	bcrypt with salt rounds for secure password storage	Auth Service
Input Validation	express-validator middleware on all endpoints	All Services
HTTPS	All services exposed via HTTPS only	All Services
CORS Policy	Restricted to known service origins only	All Services
Environment Variables	Secrets stored in .env files, never committed to GitHub	All Services

5.2 Cloud Security (AWS / Azure)

Security Feature	Description
IAM Roles	Principle of least privilege — each service has minimal required permissions only
Security Groups	Restrict inbound/outbound traffic to required ports only
Secrets Manager	AWS Secrets Manager / Azure Key Vault for managing sensitive credentials
Private Subnets	Internal services run in private subnets, only API Gateway is public
Container Isolation	Each microservice runs in its own container with no shared filesystem

5.3 DevSecOps — SonarCloud Integration

SonarCloud is integrated into every CI/CD pipeline to perform Static Application Security Testing (SAST) on every code push. This ensures:

- Code vulnerabilities are detected early in the development cycle
- Code smells and technical debt are tracked and managed
- Security hotspots are flagged before deployment
- Code coverage is measured and reported

SonarCloud free tier is sufficient for all four public repositories in this assignment.

6. Repository Structure

6.1 Folder Structure (Each Service)

```
stayease-[service]-service/
├── src/
│   ├── routes/           ← API route definitions
│   ├── controllers/      ← Business logic
│   ├── models/           ← Database schemas
│   └── middleware/       ← Auth, validation middleware
├── .github/
│   └── workflows/
│       └── ci.yml        ← GitHub Actions pipeline
├── Dockerfile            ← Container definition
├── docker-compose.yml    ← Local development setup
├── swagger.yaml          ← OpenAPI documentation
├── .env.example          ← Environment variable template
├── .gitignore
├── package.json
└── README.md
```

6.2 Required Files Checklist

File	Description	Required
Dockerfile	Container build instructions	✅ Yes
docker-compose.yml	Local multi-container setup	✅ Yes
.github/workflows/ci.yml	GitHub Actions CI/CD pipeline	✅ Yes
swagger.yaml	OpenAPI/Swagger API contract	✅ Yes
.env.example	Template for environment variables	✅ Yes
.gitignore	Ignore node_modules, .env, etc.	✅ Yes
README.md	Service description and setup guide	✅ Yes
sonar-project.properties	SonarCloud configuration	✅ Yes

7. Project Timeline

Week	Tasks	Deliverable
Week 1	Setup GitHub org, repos, folder structure, initialize Node.js project	4 repos with base structure pushed
Week 2	Build core API endpoints, database models, Dockerfile	Working local API with Docker
Week 3	GitHub Actions CI/CD pipeline, SonarCloud integration	Automated pipeline passing
Week 4	Deploy to AWS/Azure, connect services, test integrations	Live cloud deployment
Week 5	Write project report, prepare demo, integration testing	Final deliverables ready

8. Assessment Criteria Mapping

Criteria	Weight	How We Address It
Practicality & Functionality	10%	All 4 services with working endpoints deployed on cloud
DevOps Practices & Cloud	30%	GitHub Actions CI/CD, Docker, ECS/Azure deployment
Inter-service Communication	10%	Booking↔Auth, Booking↔Hotel, Booking↔Payment
Security & DevSecOps	20%	JWT, IAM roles, SonarCloud SAST scanning
Code Quality	20%	Clean code, proper structure, SonarCloud quality gate
Report & Demonstration	10%	This report + 10-min live demo with integration proof

9. Individual Report Template

Each group member must submit their own section of the report covering their assigned microservice. Use the template below:

9.1 Introduction

[Describe your microservice, its purpose, and its role in the StayEase system]

9.2 Architecture & Design

[Describe how your service is designed, what design patterns you used, and why]

9.3 API Endpoints

[List all your endpoints with request/response examples]

9.4 Inter-Service Communication

[Explain which services your service calls and which services call yours. Include code examples]

9.5 DevOps Practices

[Explain your CI/CD pipeline, Docker setup, and cloud deployment process]

9.6 Security Measures

[Explain JWT usage, IAM roles, SonarCloud results, and any other security measures]

9.7 Challenges & Solutions

[Document any challenges faced during development or integration, and how you solved them]

9.8 Conclusion

[Summarize your contribution and what you learned from the project]

10. Shared Resources & Useful Links

Resource	URL / Details
GitHub Organization	https://github.com/stayease-sliit-2026
Auth Service Repo	https://github.com/stayease-sliit-2026/stayease-auth-service
Hotel Service Repo	https://github.com/stayease-sliit-2026/stayease-hotel-service
Booking Service Repo	https://github.com/stayease-sliit-2026/stayease-booking-service
Payment Service Repo	https://github.com/stayease-sliit-2026/stayease-payment-service
MongoDB Atlas	https://www.mongodb.com/cloud/atlas (Free Tier)
Docker Hub	https://hub.docker.com (Free Tier)
SonarCloud	https://sonarcloud.io (Free for public repos)
GitHub Actions Docs	https://docs.github.com/en/actions
Swagger Editor	https://editor.swagger.io